

Los errores que convierten tu aplicación Blazor en una aplicación lenta

Y cómo arreglarlos midiendo cada cambio

Emiliano Montesdeoca del Puerto

NetCoreConf Madrid 2026 · 23 de octubre

Sponsors



‹NetCoreConf/›
Madrid '26

Platinum sponsors

Inkoova

intelequia

NTT DATA

TOKIOTA

V-Valley

Gold sponsors

axazure by avvale

plain concepts

LIDL

SCRM INTERNATIONAL HUB

Colabora

sessionize

Hola, soy Emi

- No he estado nunca en la cárcel
- Soy de Uruguay, pero muy canario
- Cloud Solutions Team Lead · Intelequia
- Microsoft MVP en Developer Technologies desde 2022
- Blogger / intento de influencer





¿Qué es Blazor?

Un framework de **ASP.NET Core** para crear interfaces web con **componentes Razor: HTML y C#**.

```
<button @onclick="Incrementar">Clics: @clics</button>

@code {
    private int clics;
    private void Incrementar() => clics++;
}
```

Cada componente tiene su interfaz, su estado y sus eventos.



¿Dónde se ejecuta una aplicación Blazor?

- **Static SSR:** el servidor genera HTML por petición.
- **Interactive Server:** C# y el renderizado, en el servidor.
  Los eventos y las actualizaciones del DOM viajan por SignalR.
- **Interactive WebAssembly:** C# y el renderizado, en el navegador.

Un componente puede renderizarse sin que cambie el DOM.



La aplicación que vamos a usar

BlazorLento: una aplicación de pedidos con un buscador y un carrito.

1. Las filas **se renderizan aunque sus datos no cambien**.
2. El buscador llama a la API **con cada tecla**.
3. La tabla carga **miles de filas que no se ven**.
4. Cambiar la vista **vuelve a consultar el mismo pedido**.
5. Añadir al carrito **hace que la tabla se vuelva a renderizar**.



ANTES / 1. Las filas se renderizan sin haber cambiado

Cada segundo, el padre se actualiza y vuelve a pasar cada pedido a su fila:

```
@foreach (var pedido in _pedidos)
{
    <FilaPedidoLenta @key="pedido.Id" Pedido="pedido" />
}
```

Cada fila recibe el mismo `Pedido`, el objeto con sus datos.

Las 200 filas se renderizan cada segundo, aunque sus datos no cambien.



DESPUÉS / El mismo hijo solo se actualiza si cambia el pedido

Ahora solo se actualiza la fila que cambia.

La misma fila comprueba si su pedido ha cambiado; el padre sigue igual:

```
private int _idRenderizado = -1;
private int _versionRenderizada = -1;

protected override bool ShouldRender()
    => Pedido.Id != _idRenderizado || Pedido.Version != _versionRenderizada;

protected override void OnAfterRender(bool firstRender)
{
    _idRenderizado = Pedido.Id;
    _versionRenderizada = Pedido.Version;
}
```



Qué significan el Id y la versión

- `Pedido` es el modelo mutable de una fila: Id, cliente, producto y estado.
- Al cambiar un dato visible, el padre incrementa `Pedido.Version`.
- El hijo guarda Id y Version tras renderizarse; solo repite el render si alguno cambia.
- También puedes pasar **parámetros primitivos** que Blazor compare.

No usar `ShouldRender => false` como receta general.



PARA TU IA / Copia este prompt

Revisa la fila de esta app Blazor: recibe un `Pedido` mutable y su padre se actualiza periódicamente. Evita renderizar las filas cuyos datos visibles no cambian, pero conserva la actualización cuando sí cambian. No uses `ShouldRender => false` como atajo. Inspecciona cómo se modifican los pedidos, implementa la opción más sencilla y verifica ambos casos.



ANTES / 2. Cada tecla del buscador llama a la API

Cada tecla del input llama a este método con `@oninput`:

```
private async Task Buscar(ChangeEventArgs e)
{
    _texto = e.Value?.ToString() ?? "";
    var texto = _texto;
    _resultados = await Api.BuscarAsync(texto);
    _textoResultados = texto;
}
```

Escribir «martinez» hace 8 llamadas a la API.

Una búsqueda anterior puede terminar después y sustituir los resultados actuales.



DESPUÉS / La búsqueda espera y cancela la anterior

Escribir «martinez» hace una sola llamada.

Espera a que dejes de escribir y cancela la búsqueda anterior:

```
var texto = _texto;
var ct = _cts.Token;

try
{
    await Task.Delay(DebounceMs, ct);
    var resultados = await Api.BuscarAsync(texto, ct);
    ct.ThrowIfCancellationRequested();
    _resultados = resultados;
}
catch (OperationCanceledException) when (ct.IsCancellationRequested) { }
```



Debounce y cancelación

- **Debounce:** espera 300 ms desde la última tecla antes de buscar.
- **Cancelación:** detiene la espera o la llamada anterior cuando cambia el texto.
- **Protección:** no muestra una respuesta si ya hay una búsqueda más reciente.
- Al salir de la página, `Dispose` cancela la operación pendiente.



PARA TU IA / Copia este prompt

> Analiza el buscador de esta app Blazor, que llama a la API en cada `oninput`. Añade debounce y cancelación de la búsqueda anterior; pasa el token a la API y evita que una respuesta antigua reemplace los resultados actuales. Cancela y libera el token al salir del componente, conserva la experiencia existente y verifica la escritura rápida, la pausa y la cancelación. No ocultes errores distintos de una cancelación esperada.



ANTES / 3. Cargar 5.000 filas para mostrar unas pocas

La tabla renderiza todos los pedidos, aunque solo se vean unos pocos:

```
@foreach (var p in _pedidos)
{
    <tr data-pedido="@p.Id">
        <td>@p.Id</td>
        <td>@p.Cliente</td>
        <td>@p.Producto</td>
        <td class="num">@p.Cantidad</td>
        <td class="num">@p.Importe</td>
        <td>@p.Estado</td>
    </tr>
}
```

5.000 filas con 6 celdas cada una: 30.000 celdas en la página.

El contenedor solo muestra unas **15 filas** a la vez.



DESPUÉS / La transformación: virtualizar la tabla

La misma tabla con `Virtualize`:

```
<Virtualize TItem="Pedido" Items="_pedidos" ItemSize="38"
            OverscanCount="5" SpacerElement="tr" Context="p">
    <tr data-pedido="@p.Id">
        <td>@p.Id</td>
        <td>@p.Cliente</td>
        <td>@p.Producto</td>
        <td class="num">@p.Cantidad</td>
        <td class="num">@p.Importe.ToString("C", CultureInfo.GetCultureInfo("es-ES"))</td>
        <td>@p.Estado</td>
    </tr>
</Virtualize>
```

Solo se renderizan las filas que ves.



Virtualizar no es paginar los datos

Implementación	Filas en el DOM	Datos solicitados
`foreach`	5.000	5.000
`Virtualize` + `Items`	Visibles + margen	5.000
`Virtualize` + `ItemsProvider`	Visibles + margen	Solo los necesarios

Renderizar menos filas y **cargar menos datos** son mejoras distintas.



PARA TU IA / Copia este prompt

Revisa esta lista Blazor que genera miles de filas con `foreach`. Sustitúyela por `Virtualize` y conserva el marcado válido dentro de la tabla. Usa `Items` si los datos ya están cargados; si la lista debe ser grande también en memoria, propone `ItemsProvider` con paginación y cancelación. Comprueba la altura real de las filas, el número visible en el DOM y que se puede recorrer hasta el final. No confundas virtualización con paginación.



ANTES / 4. Cambiar la vista vuelve a consultar el pedido

El detalle recibe el Id del pedido y si se muestra en vista compacta:

```
protected override async Task OnParametersSetAsync()
    => _pedido = await Api.GetPedidoAsync(PedidoId);
```

Activas la vista compacta del pedido 1… **y se vuelve a pedir el pedido 1 a la API.**



DESPUÉS / Consultar solo si cambia el Id

Cambiar la vista ya no llama a la API. Elegir otro pedido, sí.

```
if (PedidoId != _idSolicitado)
{
    _idSolicitado = PedidoId;
    _pedido = await Api.GetPedidoAsync(PedidoId);
}
```



PARA TU IA / Copia este prompt

Revisa este detalle Blazor: recibe `PedidoId` y `VistaCompacta`, pero vuelve a llamar a la API en cada `OnParametersSetAsync`. Cambia la lógica para consultar solo cuando cambia `PedidoId`. Conserva el cambio de presentación y la cancelación si el Id cambia durante una carga. Verifica que cambiar la vista no llama a la API y que seleccionar otro pedido sí.



ANTES / 5. Añadir al carrito también renderiza la tabla

Cuando cambia el carrito, se avisa a todos los componentes:

```
public event Action? OnChange;

public void AñadirAlCarrito(int pedidoId)
{
    Carrito.Add(pedidoId);
    OnChange?.Invoke();
}
```

El carrito y la tabla reciben el aviso, **pero la tabla no usa los datos del carrito.**



DESPUÉS / Notificar solo al componente afectado

Al añadir al carrito, la tabla ya no se vuelve a renderizar.

```
public event Action? CarritoCambiado;
public event Action? FiltroCambiado;

CarritoCambiado?.Invoke(); // Al añadir al carrito
FiltroCambiado?.Invoke(); // Al cambiar el filtro
```



PARA TU IA / Copia este prompt

En este demo Blazor, el Carrito y la Tabla se suscriben al mismo evento general. Añadir un pedido al carrito hace que la Tabla se renderice, aunque sus filas no cambien. Separa las notificaciones para que el Carrito escuche los cambios del carrito y la Tabla los del filtro. Comprueba que añadir no renderiza la Tabla y que cambiar el filtro sí.



Cinco preguntas para revisar tu aplicación

Pregunta	Qué revisar	Qué medir
¿Se renderiza sin cambios?	Parámetros y `ShouldRender`	Renderizados
¿Busco con cada tecla?	Debounce y cancelación	Llamadas
¿Renderizo filas que no se ven?	`Virtualize` y paginación	Filas y lotes
¿Cambiar la vista repite la consulta?	Cargar según el Id	Llamadas
¿Qué componentes usan el dato cambiado?	Notificaciones específicas	Renders por componente

No hace falta optimizarlo todo, sino corregir lo que está causando la lentitud.



ONE MORE TIP / 1 · Agrupa llamadas relacionadas

Idea: si la pantalla siempre necesita varios datos juntos, pídelos a un único endpoint que devuelva un DTO.

- Reduce viajes de red, no solo tiempo de espera.
- `Task.WhenAll` paraleliza, pero no reduce el número de solicitudes.

```
var panel = await http.GetFromJsonAsync<PanelDto>(
    $"/api/pedidos/{id}/panel", ct);
```



ONE MORE TIP / 2 · Cachea lecturas que cambian poco

Idea: evita volver a leer datos que siguen vigentes.

- Define una duración y cuándo invalidar.
- No compartas datos de usuario entre sesiones.

```
if (cache.TryGetValue("productos",
        out IReadOnlyList<Producto>? productos)
    && productos is not null)
    return productos;

productos = await api.GetProductosAsync(ct);
cache.Set("productos", productos, TimeSpan.FromMinutes(5));
return productos;
```



ONE MORE TIP / 3 · Trae solo los campos que muestras

Idea: para una pantalla de lectura, proyecta los campos necesarios.

- `AsNoTracking` evita seguimiento si solo vas a leer.
- Menos columnas y objetos reducen memoria y tráfico.

```
var filas = await db.Pedidos
    .AsNoTracking()
    .Where(p => p.Estado == estado)
    .Select(p => new PedidoFila(p.Id, p.Cliente, p.Importe))
    .ToListAsync(ct);
```



ONE MORE TIP / 4 · Pagina desde el servidor

Idea: en colecciones grandes, solicita una página y pide la siguiente al desplazarte.

- Usa un orden estable y limita el tamaño de página.
- Para desplazamientos muy profundos, valora paginación por cursor.

```
var pagina = await db.Pedidos
    .OrderBy(p => p.Id)
    .Skip(offset)
    .Take(pageSize)
    .Select(p => new PedidoFila(p.Id, p.Cliente, p.Importe))
    .ToListAsync(ct);
```



ONE MORE TIP / 5 · Mide antes y después

Idea: compara la misma operación con los mismos datos y condiciones.

- Mide lo que le importa al usuario y registra el volumen de datos.
- No conviertas una medición aislada en una promesa universal.

```
var inicio = Stopwatch.GetTimestamp();
var pedidos = await api.GetPedidosAsync(200, ct);
logger.LogInformation(
    "Pedidos={Count}; {ElapsedMs} ms",
    pedidos.Count,
    Stopwatch.GetElapsedTime(inicio).TotalMilliseconds);
```

Sponsors



‹NetCoreConf/›
Madrid '26

Platinum sponsors

Inkoova

intelequia

NTT DATA

TOKIOTA

V-Valley

Gold sponsors

axazure by avvale

plain concepts

LIDL

SCRM INTERNATIONAL HUB

Colabora

sessionize



Gracias

¿Preguntas?

Emiliano Montesdeoca
linkedin.com/in/emimontesdeoca · @emimontesdeocaa